

Artificial Intelligence

5. Multilayer Perceptrons

RUPP. MITE

Cohort 20. Year 1. Semester 1

2026.03.16 ~ 2026.07.19

Rim Beanbonyka, PhD

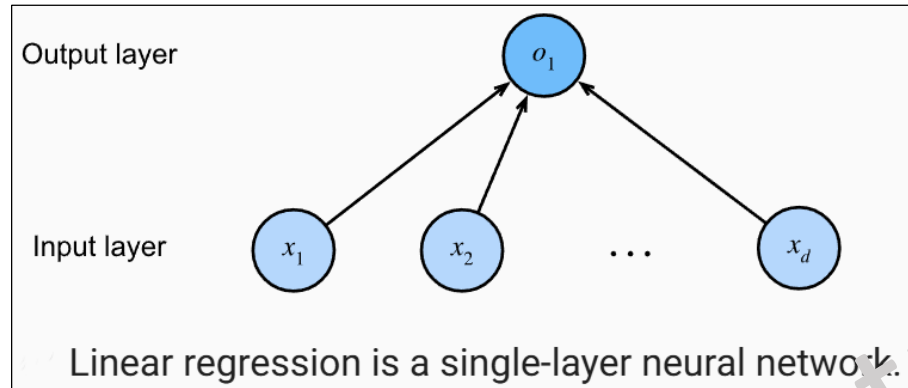
1. Multilayer perceptrons
2. Implementation of multilayer perceptrons
3. Forward propagation, backward propagation, and computational graphs
4. Numerical stability and initialization
5. Generalization in Deep Learning
6. Dropout
7. Predicting house prices on Kaggle

1. Multilayer perceptrons

- Limitations of Linear models:

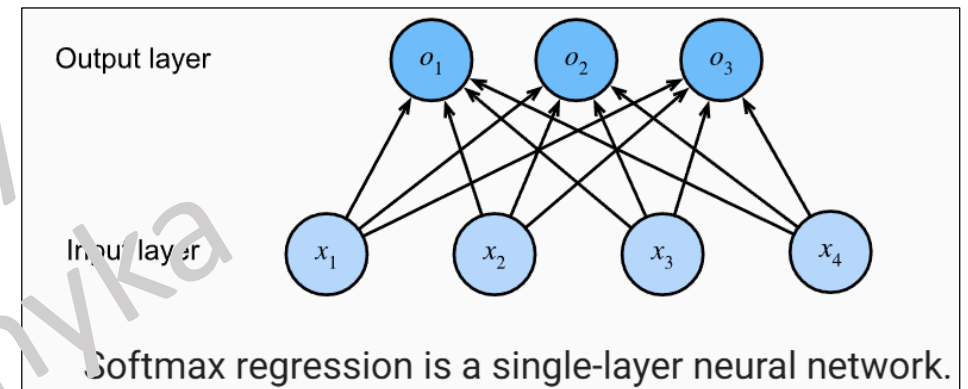
- Linear regression model:

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$



- Linear regression model:

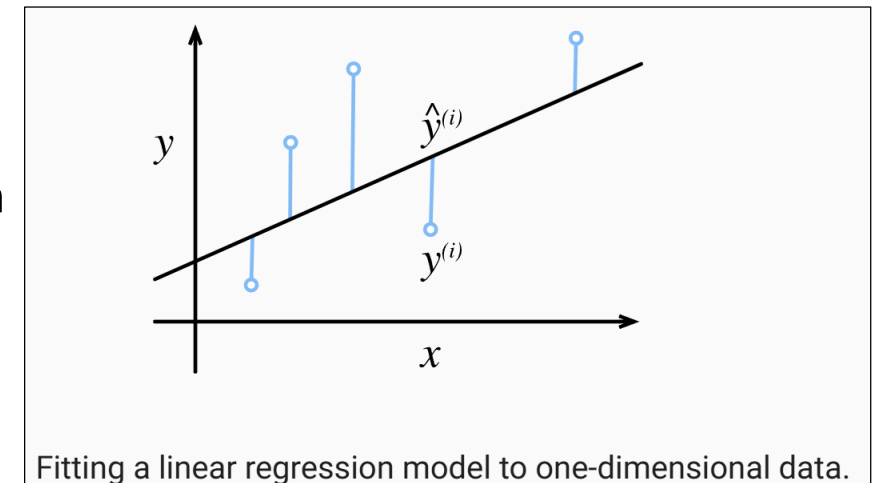
$$\mathbf{O} = \mathbf{XW} + \mathbf{b},$$
$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{O})$$



- Linearity implies the weaker assumption of **monotonicity**,

- i.e., that any **increase** in our feature must either always cause an **increase** in our model's output (if the corresponding weight is **positive**), or

- always cause a **decrease** in our model's output (if the corresponding weight is **negative**).

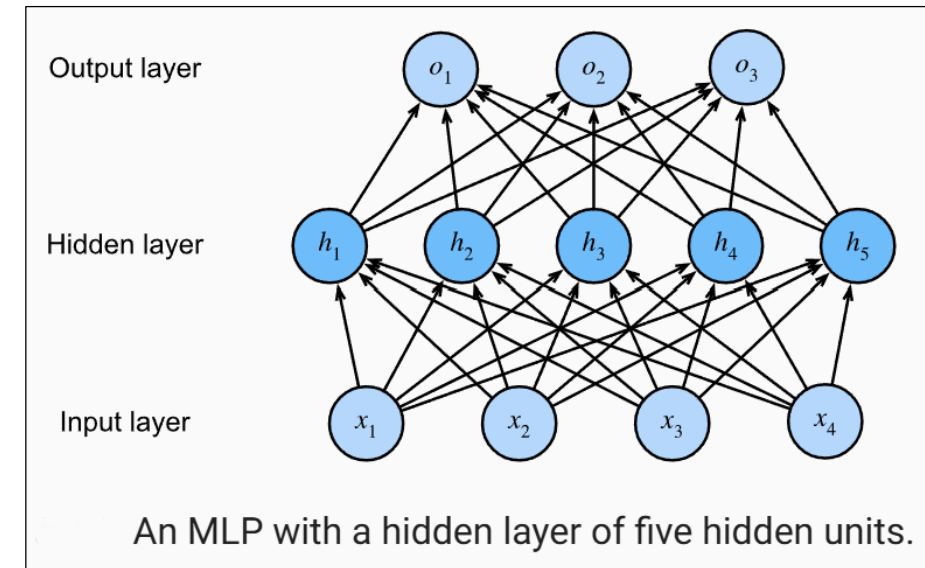


1. Multilayer perceptrons

- Hidden Layers:

- We can overcome the limitations of linear models by incorporating **one or more hidden layers**.
- The easiest way to do this is to **stack** many fully connected layers on top of one another.
- Each layer feeds into the **layer above it**, until we generate outputs.
- We can think of the first layers as our **representation** and the final layer as our **linear predictor**.
- This architecture is commonly called a **multilayer perceptron**, often abbreviated as **MLP**.

- This MLP has **four** inputs, **three** outputs, and its hidden layer contains **five** hidden units.
- Since the **input layer** does not involve any calculations, producing outputs with this network requires implementing the computations for both the **hidden and output layers**; thus, the number of layers in this MLP is **two**.
- Note that both layers are **fully connected**.
- Every input **influences** every neuron in the hidden layer, and each of these in turn influences every neuron in the output layer.



1. Multilayer perceptrons

- From Linear to NonLinear model:

$$\begin{array}{l} \mathbf{H} = \mathbf{XW}^{(1)} + \mathbf{b}^{(1)} \\ \mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)} \end{array} \quad \begin{array}{lll} \mathbf{X} \in \mathbb{R}^{n \times d} & \mathbf{W}^{(1)} \in \mathbb{R}^{d \times h} & \mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h} \\ \mathbf{H} \in \mathbb{R}^{n \times h} & \mathbf{W}^{(2)} \in \mathbb{R}^{h \times q} & \mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q} \quad \mathbf{O} \in \mathbb{R}^{n \times q} \end{array}$$

- In the model defined above—we gain nothing for our troubles!

$$\mathbf{O} = (\mathbf{XW}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW} + \mathbf{b}.$$

- In order to transform from linear to nonlinear model, a nonlinear activation function to be applied to each hidden unit following the affine transformation.

$$\begin{array}{l} \mathbf{H} = \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}) \\ \mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)} \end{array}$$

- The outputs of activation functions $\sigma(\cdot)$ are called *activations*.
- A popular choice is the ReLU (rectified linear unit) activation function

$$\begin{array}{l} \mathbf{H}^{(1)} = \sigma_1(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}) \\ \mathbf{H}^{(2)} = \sigma_2(\mathbf{H}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}) \\ \vdots \end{array}$$

$$\sigma(x) = \max(0, x)$$

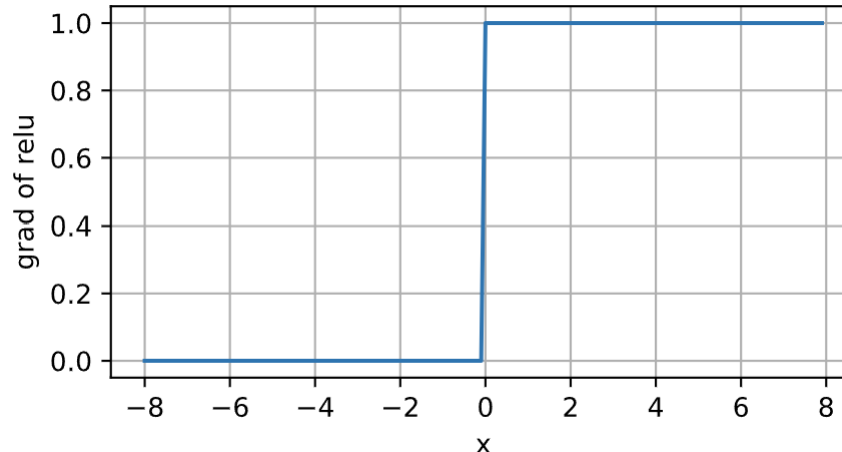
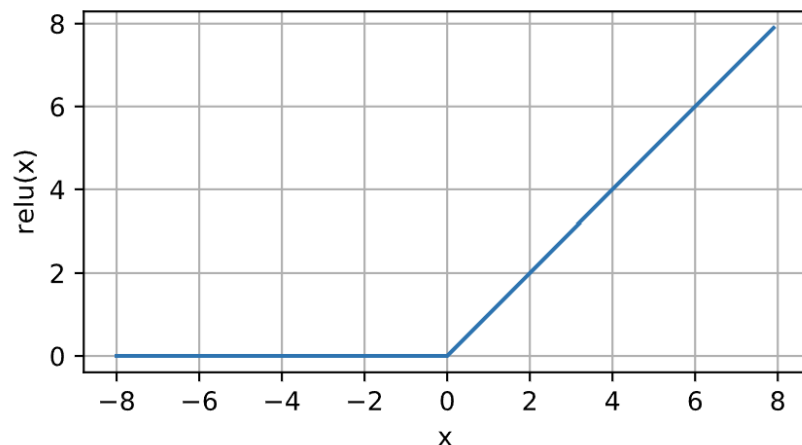
- Activation functions:

- Activation functions decide whether a neuron should be **activated** or not before **further** calculate the weighted sum and adding bias to it.

1. ReLU function

- The most popular choice, due to both simplicity of implementation and its good performance on a variety of predictive tasks, is the rectified linear unit (ReLU).
- ReLU provides a very simple nonlinear transformation.
- Given an element x , the function is defined as the maximum of that element and 0 :

$$\text{ReLU}(x) = \max(x, 0)$$



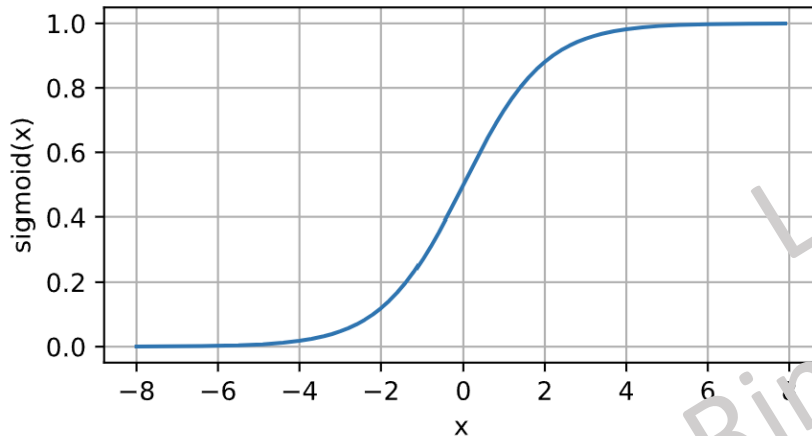
1. Multilayer perceptrons

p.7

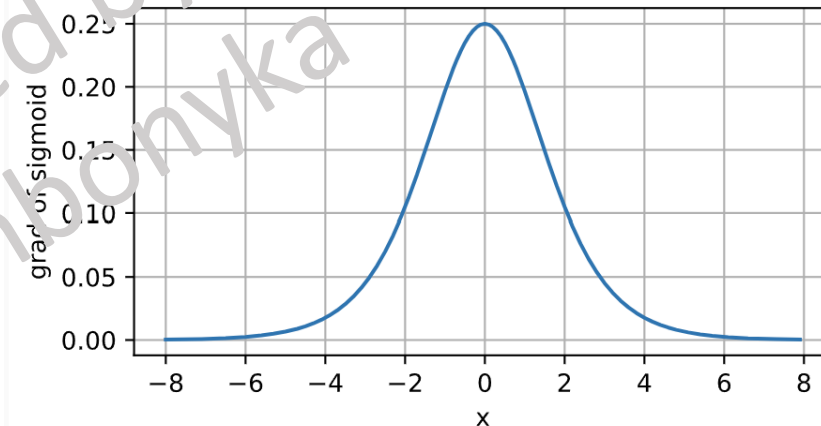
2. Sigmoid function

- The sigmoid function transforms those inputs whose values lie in the domain $\mathbb{R}$, to outputs that lie on the interval $(0, 1)$.
- For that reason, the sigmoid is often called a squashing function: it squashes any input in the range $(-\infty, \infty)$ to some value in the range $(0, 1)$:

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$



$$\frac{d}{dx} \text{sigmoid}(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2} = \text{sigmoid}(x) (1 - \text{sigmoid}(x))$$



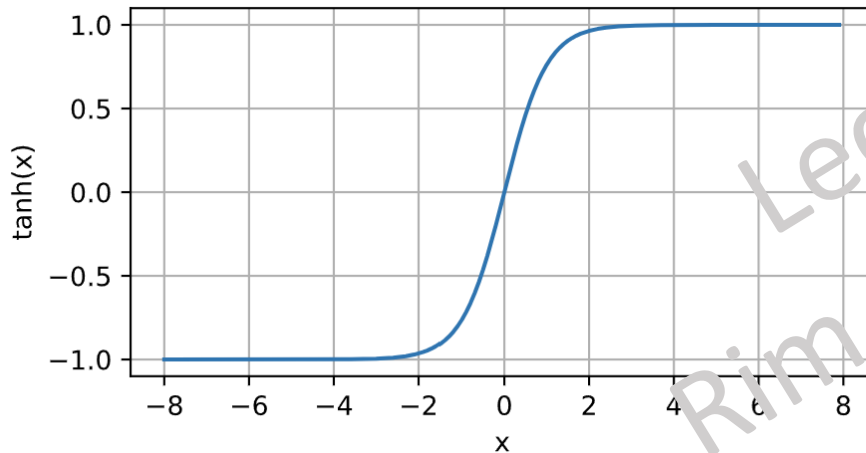
- When attention shifted to gradient-based learning, the sigmoid function was a natural choice because it is a smooth, differentiable approximation to a thresholding unit.
- Sigmoids are still widely used as activation functions on the output units when we want to interpret the outputs as probabilities for binary classification problems: you can think of the sigmoid as a special case of the softmax.

1. Multilayer perceptrons

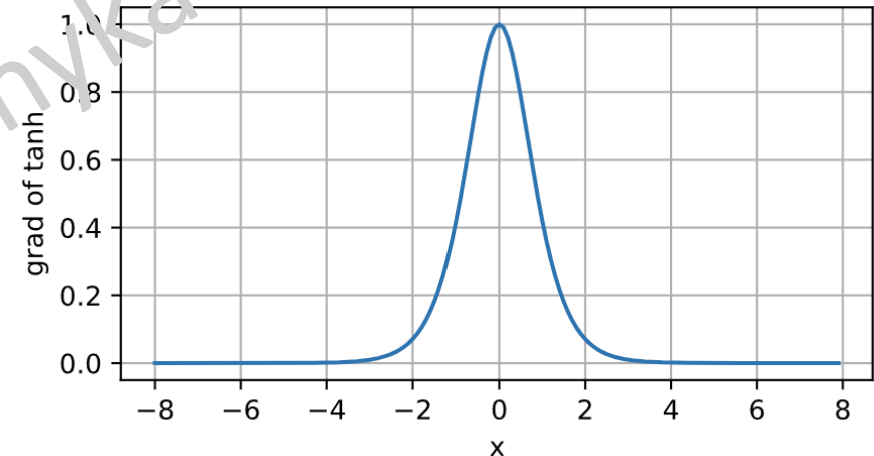
3. Tanh function

- Like the sigmoid function, the tanh (hyperbolic tangent) function also squashes its inputs, transforming them into elements on the interval between -1 and +1:

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$



- It is plotted below. As the input nears 0, the derivative of the tanh function approaches a maximum of 1.
- And as we saw with the sigmoid function, as input moves away from 0 in either direction, the derivative of the tanh function approaches 0.

- So far, we have trained our models, we only worried about the calculations involved in **forward propagation** through the model.
- When it came time to calculate the gradients, we just invoked the **backward propagation** function provided by the deep learning framework: `y.backward()`.

1. Forward propagation

- Forward propagation (or forward pass) refers to **the calculation and storage** of intermediate variables (including outputs) for a neural network in order from the input layer to the output layer.

- Input $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{W}^{(1)} \in \mathbb{R}^{h \times d}$, no bias for simplicity:

- Intermediate variable $\mathbf{z} \in \mathbb{R}^h$:

$$\mathbf{z} = \mathbf{W}^{(1)} \mathbf{x}$$

- Hidden variable $\mathbf{h}$:

$$\mathbf{h} = \phi(\mathbf{z})$$

- Output variable $\mathbf{o}$:

$$\mathbf{o} = \mathbf{W}^{(2)} \mathbf{h}$$

- Loss variable L :

$$L = l(\mathbf{o}, y)$$

- ℓ_2 regularization variable s :

$$s = \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2 \right)$$

- Objective variable J :

$$J = L + s.$$

A one-hidden-layer MLP with weight decay (ℓ_2 regularization)

$$\mathbf{H} = \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)})$$

$$\mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}$$

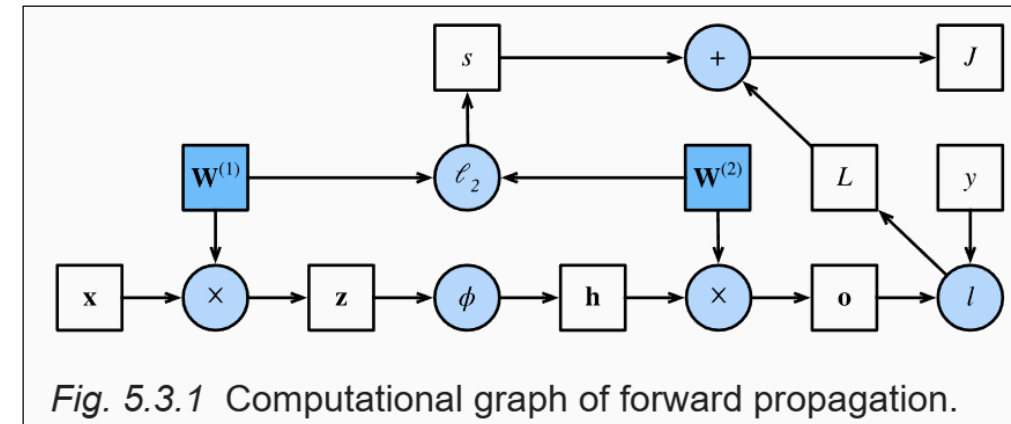


Fig. 5.3.1 Computational graph of forward propagation.

2. Backward propagation (backpropagation)

- Backpropagation refers to the method of calculating the **gradient** of neural network parameters.
- In short, the method traverses the network in **reverse order**, from the output to the input layer, according to the **chain rule** from calculus.
- The objective of backpropagation is to calculate the gradients $\frac{\partial J}{\partial \mathbf{W}^{(1)}}$ and $\frac{\partial J}{\partial \mathbf{W}^{(2)}}$.

- Gradient of objective respect to loss and regularization: $\frac{\partial J}{\partial L} = 1$ and $\frac{\partial J}{\partial s} = 1$
- Gradient of objective respect to output $\mathbf{o}$: $\frac{\partial J}{\partial \mathbf{o}} = \text{prod} \left(\frac{\partial J}{\partial L}, \frac{\partial L}{\partial \mathbf{o}} \right) = \frac{\partial L}{\partial \mathbf{o}} \in \mathbb{R}^q$
- Gradient of regularization respect to both parameters : $\frac{\partial s}{\partial \mathbf{W}^{(1)}} = \lambda \mathbf{W}^{(1)}$ and $\frac{\partial s}{\partial \mathbf{W}^{(2)}} = \lambda \mathbf{W}^{(2)}$
- Gradient of objective respect to parameter $\mathbf{W}^{(2)}$: $\frac{\partial J}{\partial \mathbf{W}^{(2)}} = \text{prod} \left(\frac{\partial J}{\partial \mathbf{o}}, \frac{\partial \mathbf{o}}{\partial \mathbf{W}^{(2)}} \right) + \text{prod} \left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial \mathbf{W}^{(2)}} \right) = \frac{\partial J}{\partial \mathbf{o}} \mathbf{h}^\top + \lambda \mathbf{W}^{(2)}$
- Gradient of objective respect to hidden $\mathbf{h}$: $\frac{\partial J}{\partial \mathbf{h}} = \text{prod} \left(\frac{\partial J}{\partial \mathbf{o}}, \frac{\partial \mathbf{o}}{\partial \mathbf{h}} \right) = \mathbf{W}^{(2)\top} \frac{\partial J}{\partial \mathbf{o}}$
- Gradient of objective respect to Intermediate $\mathbf{z}$: $\frac{\partial J}{\partial \mathbf{z}} = \text{prod} \left(\frac{\partial J}{\partial \mathbf{h}}, \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \right) = \frac{\partial J}{\partial \mathbf{h}} \odot \phi'(\mathbf{z})$
- Gradient of objective respect to parameter $\mathbf{W}^{(1)}$: $\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \text{prod} \left(\frac{\partial J}{\partial \mathbf{z}}, \frac{\partial \mathbf{z}}{\partial \mathbf{W}^{(1)}} \right) + \text{prod} \left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial \mathbf{W}^{(1)}} \right) = \frac{\partial J}{\partial \mathbf{z}} \mathbf{x}^\top + \lambda \mathbf{W}^{(1)}.$

- Thus far, every model that we have implemented required that we **initialize its parameters** according to some **pre-specified distribution**.
- Until now, we took the initialization scheme for granted, glossing over the details of **how** these choices are made.
- The choice of initialization scheme plays a significant role in neural network learning, and it can be crucial for maintaining **numerical stability**.
- Moreover, these choices can be tied up in interesting ways with the choice of the **nonlinear activation function**.
- Which function we choose and how we initialize parameters can determine how quickly our **optimization algorithm converges**.
- Poor choices here can cause us to encounter **exploding** or **vanishing gradients** while training.

❖ Vanishing and Exploding Gradients:

- We may be facing parameter updates that are either
 - (i) excessively **large**, destroying our model (the **exploding** gradient problem); or
 - (ii) excessively **small** (the **vanishing** gradient problem), rendering learning impossible as parameters hardly move on each update.

5. Generalization in Deep Learning

p.12

- Our real quest all along was to discover **general patterns** on the basis of which we can make accurate predictions even on **new examples** drawn from the same underlying population.
- At its core, deep learning is a **statistical discipline** and we wish to **optimize training loss** only insofar as some statistical principle (known or unknown) leads the resulting models to **generalize beyond the training set**.

❖ Overfitting and Regularization:

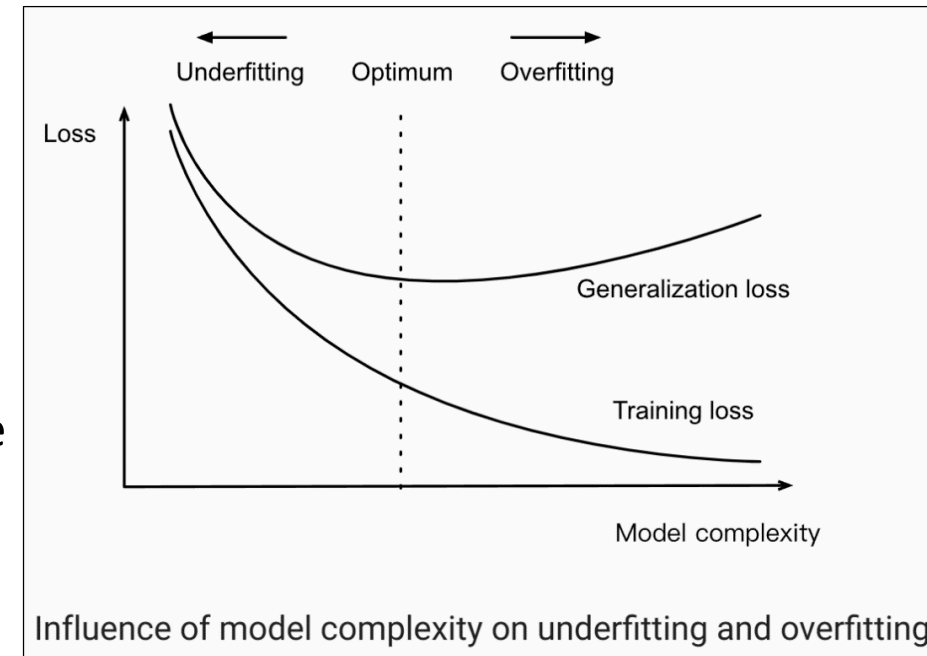
- With deep learning models encoding inductive biases, our approach to training them typically consists of two phases:

(i) **fit** the training data; and

(ii) estimate the **generalization error** (the true error on the underlying population) by evaluating the model on holdout data.

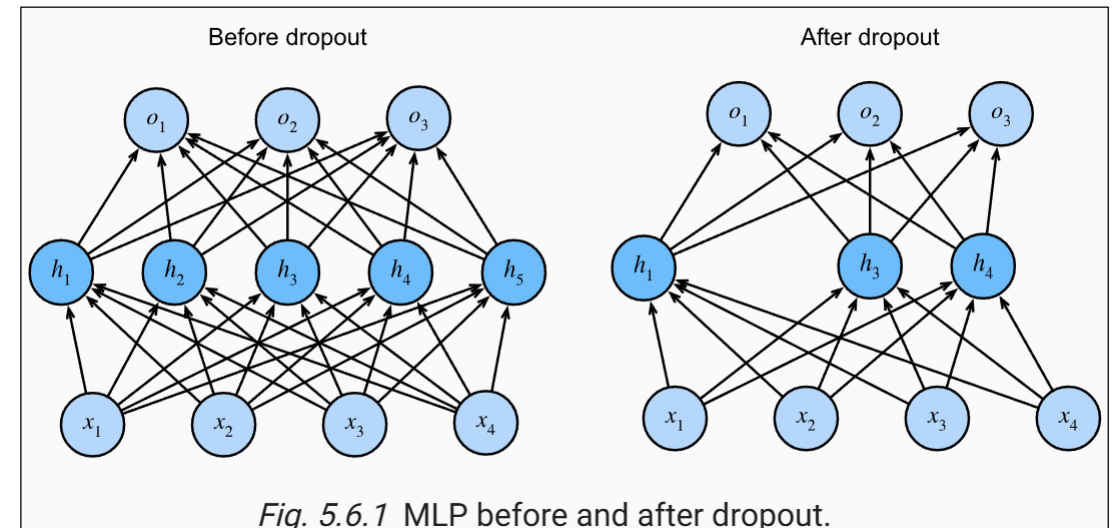
❖ Early stopping:

- The most common way to determine the **stopping criterion** is to monitor **validation error** throughout training (typically by checking once after each epoch) and to **cut off training** when the validation error has **not decreased** by more than some **small amount ϵ** for some number of epochs.
- This is sometimes called a **patience criterion**.



- Let's think briefly about what we expect from a **good predictive model**.
- We want it to perform well on **unseen data**.
- Classical generalization theory suggests that to **close the gap** between train and test performance, we should aim for a **simple model**. Simplicity can come in the form of a **small number of dimensions**.
- As we saw when discussing **weight decay** (regularization), the (inverse) norm of the parameters also represents a useful measure of simplicity.
- Another method is called **dropout** regularization:
 - First, **zero out** some fraction of the nodes in each layer;
 - and then, debias each layer by **normalizing** by the fraction of nodes that were retained (not dropped out).

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$



Good Luck!

